```
#!/usr/bin/env python3
"""
CTF Encoder/Decoder Toolkit
A menu-driven script for common encoding/decoding operations used in CTFs.
"""

import base64
import hashlib
import urllib.parse
import html
import sys

# ------------------------------------------------
# BASE64
# ------------------------------------------------
def base64_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = base64.b64encode(text.encode()).decode()
    elif action == "2":
        try:
            result = base64.b64decode(text).decode()
        except Exception:
            result = "[!] Invalid Base64 input"
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------------
# BASE32
# ------------------------------------------------
def base32_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = base64.b32encode(text.encode()).decode()
    elif action == "2":
        try:
            result = base64.b32decode(text).decode()
        except Exception:
            result = "[!] Invalid Base32 input"
    else:
```

```
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# BASE58
# ------------------------------------------
BASE58_ALPHABET = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz"

def base58_encode(data: bytes) -> str:
    num = int.from_bytes(data, "big")
    result = ""
    while num > 0:
        num, rem = divmod(num, 58)
        result = BASE58_ALPHABET[rem] + result
    for byte in data:
        if byte == 0:
            result = BASE58_ALPHABET[0] + result
        else:
            break
    return result

def base58_decode(s: str) -> bytes:
    num = 0
    for char in s:
        if char not in BASE58_ALPHABET:
            raise ValueError(f"Invalid character: {char}")
        num = num * 58 + BASE58_ALPHABET.index(char)
    result = []
    while num > 0:
        num, rem = divmod(num, 256)
        result.insert(0, rem)
    for char in s:
        if char == BASE58_ALPHABET[0]:
            result.insert(0, 0)
        else:
            break
    return bytes(result)

def base58_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = base58_encode(text.encode())
    elif action == "2":
        try:
            result = base58_decode(text).decode()
```

```
        except Exception as e:
            result = f"[!] Error: {e}"
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# HEX
# ------------------------------------------
def hex_tool():
    action = input("  [1] Encode (text to hex)  [2] Decode (hex to text) : ").str
    text = input("  Enter text: ").strip()
    if action == "1":
        result = text.encode().hex()
    elif action == "2":
        try:
            result = bytes.fromhex(text).decode()
        except Exception:
            result = "[!] Invalid hex input"
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# BINARY
# ------------------------------------------
def binary_tool():
    action = input("  [1] Text to Binary  [2] Binary to Text : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = " ".join(format(ord(c), "08b") for c in text)
    elif action == "2":
        try:
            bits = text.split()
            result = "".join(chr(int(b, 2)) for b in bits)
        except Exception:
            result = "[!] Invalid binary input (use space-separated 8-bit groups)
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# ROT13
# ------------------------------------------
```

```
def rot13_tool():
    text = input("  Enter text: ").strip()
    result = text.translate(str.maketrans(
        "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz",
        "NOPQRSTUVWXYZABCDEFGHIJKLMnopqrstuvwxyzabcdefghijklm"
    ))
    print(f"\n  Result: {result}")


# ------------------------------------------
# CAESAR CIPHER (brute force all 25 shifts)
# ------------------------------------------
def caesar_tool():
    text = input("  Enter text to brute force: ").strip()
    print("\n  All 25 shifts:")
    for shift in range(1, 26):
        result = ""
        for char in text:
            if char.isalpha():
                base = ord("A") if char.isupper() else ord("a")
                result += chr((ord(char) - base + shift) % 26 + base)
            else:
                result += char
        print(f"  Shift {shift:>2}: {result}")


# ------------------------------------------
# URL ENCODE / DECODE
# ------------------------------------------
def url_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        result = urllib.parse.quote(text)
    elif action == "2":
        result = urllib.parse.unquote(text)
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# HTML ENTITY ENCODE / DECODE
# ------------------------------------------
def html_tool():
    action = input("  [1] Encode  [2] Decode : ").strip()
    text = input("  Enter text: ").strip()
```

```
    if action == "1":
        result = html.escape(text)
    elif action == "2":
        result = html.unescape(text)
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# XOR
# ------------------------------------------
def xor_tool():
    text = input("  Enter text: ").strip()
    key = input("  Enter key (single char or string): ").strip()
    if not key:
        print("\n  [!] Key cannot be empty")
        return
    key_repeated = (key * (len(text) // len(key) + 1))[:len(text)]
    result = "".join(chr(ord(c) ^ ord(k)) for c, k in zip(text, key_repeated))
    print(f"\n  Result (raw) : {result}")
    print(f"  Result (hex) : {result.encode().hex()}")


# ------------------------------------------
# MORSE CODE
# ------------------------------------------
MORSE_CODE = {
    "A": ".-",   "B": "-...", "C": "-.-.", "D": "-..",
    "E": ".",    "F": "..-.", "G": "--.",  "H": "....",
    "I": "..",   "J": ".---", "K": "-.-",  "L": ".-..",
    "M": "--",   "N": "-.",   "O": "---",  "P": ".--.",
    "Q": "--.-", "R": ".-.",  "S": "...",  "T": "-",
    "U": "..-",  "V": "...-", "W": ".--",  "X": "-..-",
    "Y": "-.--", "Z": "--..",
    "0": "-----", "1": ".----", "2": "..---", "3": "...--",
    "4": "....-", "5": ".....", "6": "-....", "7": "--...",
    "8": "---..", "9": "----.",
    " ": "/"
}
REVERSE_MORSE = {v: k for k, v in MORSE_CODE.items()}

def morse_tool():
    action = input("  [1] Text to Morse  [2] Morse to Text : ").strip()
    text = input("  Enter text: ").strip()
    if action == "1":
        try:
```

```
            result = " ".join(MORSE_CODE[c.upper()] for c in text if c.upper() in
        except Exception:
            result = "[!] Unsupported characters"
    elif action == "2":
        try:
            words = text.split(" / ")
            decoded_words = []
            for word in words:
                decoded_words.append("".join(REVERSE_MORSE[code] for code in word
            result = " ".join(decoded_words)
        except Exception:
            result = "[!] Invalid morse input"
    else:
        result = "[!] Invalid option"
    print(f"\n  Result: {result}")


# ------------------------------------------
# HASH GENERATOR
# ------------------------------------------
def hash_tool():
    text = input("  Enter text to hash: ").strip()
    print(f"\n  MD5    : {hashlib.md5(text.encode()).hexdigest()}")
    print(f"  SHA1   : {hashlib.sha1(text.encode()).hexdigest()}")
    print(f"  SHA256 : {hashlib.sha256(text.encode()).hexdigest()}")
    print(f"  SHA512 : {hashlib.sha512(text.encode()).hexdigest()}")


# ------------------------------------------
# HASH IDENTIFIER
# ------------------------------------------
def hash_identifier():
    h = input("  Paste hash: ").strip()
    length = len(h)
    print()
    if length == 32:
        print("  Likely: MD5")
    elif length == 40:
        print("  Likely: SHA1")
    elif length == 56:
        print("  Likely: SHA224")
    elif length == 64:
        print("  Likely: SHA256")
    elif length == 96:
        print("  Likely: SHA384")
    elif length == 128:
        print("  Likely: SHA512")
```

```
    else:
        print(f"  Unknown hash type (length: {length})")
    is_hex = all(c in "0123456789abcdefABCDEF" for c in h)
    print(f"  Characters: {'hex only' if is_hex else 'non-hex detected'}")


# ------------------------------------------
# NUMBER BASE CONVERTER (any base 2-36)
# ------------------------------------------
def base_converter():
    print("  Convert a number between any bases (2 to 36)")
    try:
        from_base = int(input("  From base: ").strip())
        to_base   = int(input("  To base  : ").strip())

        if not (2 <= from_base <= 36) or not (2 <= to_base <= 36):
            print("\n  [!] Base must be between 2 and 36")
            return

        number = input(f"  Enter number (base {from_base}): ").strip().upper()

        # Step 1: convert input to decimal
        decimal = int(number, from_base)

        # Step 2: convert decimal to target base
        digits = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
        if decimal == 0:
            result = "0"
        else:
            result = ""
            n = decimal
            while n > 0:
                result = digits[n % to_base] + result
                n //= to_base

        print(f"\n  {number} (base {from_base}) = {result} (base {to_base})")
        print(f"  Decimal value: {decimal}")

    except ValueError as e:
        print(f"\n  [!] Error: {e}")


# ------------------------------------------
# MENU
# ------------------------------------------
MENU = """
+=====================================+
```

```
|      CTF ENCODER/DECODER TOOLKIT     |
+======================================+

  ENCODING / DECODING
  -------------------------------------
   1.  Base64
   2.  Base32
   3.  Base58
   4.  Hex
   5.  Binary
   6.  ROT13
   7.  Caesar (brute force)
   8.  URL Encode/Decode
   9.  HTML Entity Encode/Decode
   10. XOR

  MISC
  -------------------------------------
   11. Morse Code
   12. Hash Generator (MD5/SHA1/SHA256/SHA512)
   13. Hash Identifier
   14. Number Base Converter (any base 2-36)

   0.  Exit
"""

TOOLS = {
    "1":  base64_tool,
    "2":  base32_tool,
    "3":  base58_tool,
    "4":  hex_tool,
    "5":  binary_tool,
    "6":  rot13_tool,
    "7":  caesar_tool,
    "8":  url_tool,
    "9":  html_tool,
    "10": xor_tool,
    "11": morse_tool,
    "12": hash_tool,
    "13": hash_identifier,
    "14": base_converter,
}

def main():
    while True:
        print(MENU)
        choice = input("  Select option: ").strip()
```

```
        if choice == "0":
            print("\n  Goodbye!\n")
            sys.exit(0)
        elif choice in TOOLS:
            print()
            TOOLS[choice]()
            input("\n  [Press Enter to continue]")
        else:
            print("\n  [!] Invalid option, try again.")

if __name__ == "__main__":
    main()
```